6. Page Setup Guide

Contents

Shapes 1
Columns 1
Pages 2
Dimensions 2
Headers and footers 2
Advanced programming 3
Questions 4

Shapes

`1fr` acts like a spring in `#h()` and `#v()` pushing content to the right or bottom limit.

You can use set rule to set defaults for `#rect(), #circle(), #box()` and `#columns()` functions. Setting `#columns()` you affect page columns as well.

`#rect(width: 100%, height: 100%)` – makes the rectangle to fill all the available space.

`#circle(radius: ⟨length⟩)` is a function similar to `#rect()`, it also has `fill, stroke` and `inset` arguments and can wrap `[content]`.

`inset: ⟨length⟩` is an argument of `#box(), #rect() and #circle()` that expands inner space: `inset: 0pt` → `inset: 5pt`

`⟨color⟩.lighten(⟨num⟩%)` is method that makes color lighter when `⟨num⟩` is positive and darker when `⟨num⟩` is negative.

Columns

`#columns(⟨number⟩)[content]` creates a content divided into `⟨number⟩` of columns.

`#colbreak()` is like `#pagebreak()` but for columns.

`gutter: ⟨length⟩` – an argument of `#columns()`, `#table()` and `#grid()` setting a gap between columns.

Pages

`#page(fill: ⟨color⟩)` – colors the whole page.

Dimensions

`page.paper` can be a positional argument: `#set page("a6")`.

`#page(width: ⟨length⟩, height: ⟨length⟩)` – can be used instead of `page.paper`.

`#page(margin: (inside: ⟨length⟩, outside: ⟨length⟩), binding: ⟨right ∨ left⟩)` – set alternating margins.

`#page(flipped: true)` – flips the page (changes dimensions the other way around).

Headers and footers

`#page(numbering: ⟨str⟩)` – makes pages numbered. The *numbering pattern* `⟨str⟩` can contain *counting symbols* (e.g., `1, a, i, I`) and not. First counting symbol stands for the current page, the second – for the total amount of pages. Not counting ones are displayed as they are.

`#page(..., number-align: ⟨v-align + <h-align>)` puts the numbering into header or footer.

`#page(header: [...], footer: [...])` – make custom header and footer (suppress the previous 2 arguments). Use code block for fine-tuning:

```
...header: context {
    let ...
    if ... [...]
    ...}
```

`#page(..., header-ascent: ⟨length⟩, footer-descent: ⟨length⟩)` – push header/footer away from a margin edge. The `⟨length⟩` can be relative `⟨num⟩%` meaning percent of margin.

Advanced programming

`#str(⟨num⟩)` – takes a number and returns a string. A string can be used in tables and grids as well as `[...]`.

`#(...)` – allows operations with arrays:

- `#(("x",) + ("y",))` → `("x", "y")`
- `#(3 * ("x",))` → `("x", "x", "x")`

`.join(...)` – concatenates arrays with argument between elements:

- `#("a", "b", "c").join()` → abc
- `#("a", "b", "c").join("-")` → a-b-c
- `#("a", "b", "c").join(h(1em))` → a b c

`.any(x => <condition with x>)` – `true` if at least one element satisfies the condition, `false` otherwise:

- `#(1, 2, 3).any(num => num < 2)` → `true`
- `#(1, 2, 3).any(hui => hui >= 5)` → `false`

`#let (⟨name1⟩, ⟨name2⟩, ⟨name3⟩...) = (⟨val1⟩, ⟨val2⟩, ⟨val3⟩...)` – allows to declare several variables at once:

- `#let (a,) = (2,)`
  - `#a` → 2
- `let (a1, a2, a3) = ("I", "love", "you")`
  - `#a1` → “I”; `#a2` → “love”, `#a3` → “you”

`.first()` – returns first element of an array:

- `#(3, 2, 1).first()` → 3

If body of `#for` creates an array in each iteration, `#for` concatenates all these arrays. The array can be spread with `..for`:

- `#for x in (1, 2, 3) {(x, x*2)}` → `(1, 2, 2, 4, 3, 6)`
- `#table(columns: 4,`
  `..for x in (1, 2, 3, 4){(str(x),)})` →

1	2	3	4

`#context` is used with things that require “introspection”. `#context` can be used before a code block `{...}` which contains such a thing.

Counter A data type for numbered sequences. Can belong to pages, headings, figures. Counters are not arrays!

Location A data type representing a place in the document.

Element A kind of content that can be labeled. Every element has a location.

`#context query(<label>)` – returns an array of elements labeled by the label.

⟨`element`⟩`.location()` – returns a location of the element.

`#context here()` – returns location of this place.

⟨`location`⟩`.page()` – returns page of the location (doesn't depend on page counter).

`#counter(page)` – returns counter of the current page.

`.get()` – transforms a counter into an array.

`.at(`⟨`location`⟩`)` – like `.get` but looks at the value of counter in the location.

`.update(`⟨`num`⟩`)` – switches counter to the number. `.update(`⟨`closure`⟩`)` – switches counter according to the closure.

To display page numbering in a custom footer/header, we can use `#context counter(page).display(`⟨`pattern string`⟩`, both: true)`

Questions

1. How to turn a number into string?
2. Explain the role of `#()`
3. Explain the result of the code: `#let (a, ) = ("a",)`
4. What's the role of `1fr` in `#h()` and `#v()`?
5. How can we make new defaults for: `#rect, #circle(), #box(), #columns()`? For each of the functions tell possible new defaults.
6. Will `#set columns(...)` affect the columns generated by `#set page(columns...)`?
7. Explain `#pagebreak()` function.
8. How to make a rectangle to fill all the available space?

9. Explain `#circle()` function.
10. Explain `inset: ...` argument of functions.
11. Explain `gutter: ...` argument of functions.
12. Explain `.lighten()` method.
13. Explain `#columns()` function.
14. Explain `#colbreak()` function.
15. How to choose color of the page?
16. Which argument of `#page()` is positional?
17. How to make a page of custom dimensions?
18. Explain `binding: ...` argument.
19. Explain `inside: ...` and `outside: ...` sub-arguments.
20. Explain `#page(numbering: ...)` argument. Explain numbering pattern string.
21. Why `#page(numbering: "Page 1 of 1")` argument won't work as intended? How to achieve the desired effect?
22. Explain `number-align: ...` argument.
23. Explain `header: ...` and 'footer: ...' arguments.
24. Find and explain what's wrong:

```
#page("a6",
  numbering: "1",
  number-align: top + right,
  header: align(left)[#context document.title]
)[...]
```

25. How a code block can help us fine-tune behavior of headers and footers.
26. Explain `header-ascent: ...` and `footer-descent: ...` arguments. Which two different types of values they can take? How each of the types works?
27. Explain `#str()` function.
28. Find the output of the code:
    - `#(("x",) + ("y",))`
    - `#(3 * ("x",))`
29. Explain `.join()` method.
30. Explain `.any()` method.
31. How to declare several variables with one `#let`?

32. Explain `.first()` method.
33. Explain the result of the code:
    - `#for x in (1, 2, 3) {(x, x*2)}`
    - ```
      #table(columns: 4,
      ..for x in (1, 2, 3, 4){(str(x),)})
      ```
34. What is required when Typst needs "introspection" of the documents?
35. Explain the functionality of `#context {...}` construction.
36. What are a counter, a location and an element as Typst entities?
37. Explain `query()` function.
38. Explain `.location()` method.
39. Explain `here()` function.
40. Explain `.page()` method,.
41. Explain `#counter(page)`.
42. Explain `.get()` and `.at()`.
43. Explain `.update()`.
44. Explain .display().
45. Explain `flipped: ...` argument.
      ```